



Estoy construyendo ArchiCheck, una herramienta que analiza planos arquitectónicos en PDF (2D, no BIM/DWG) para pre-validar el cumplimiento de normativa de construcción chilena (OGUC, LGUC, DDU, PRC comunales) antes de presentar un proyecto a la Dirección de Obras Municipales. El público objetivo son estudios de arquitectura chilenos, en su mayoría pequeños, que hoy trabajan en PDF, no en Revit/ArchiCAD.

Qué ya construimos (para que no me recomiendes reinventar esto)

El pipeline actual corre en Python/Colab y tiene dos capas:

1. **Extracción geométrica determinística** (PyMuPDF + OpenCV): parseo directo de los objetos vectoriales del PDF (`get_drawings()` — líneas, curvas Bézier, rectángulos, quads), agrupamiento por conectividad (Union-Find) para reconstruir muros como cadenas de segmentos, ajuste de círculo (método de Kasa) para detectar arcos de puerta, clasificación de puertas también desde curvas Bézier reales del PDF, exclusión de achurado/ejes/cotas por color y geometría, y segmentación de recintos vía `adaptiveThreshold` + `connectedComponentsWithStats`.
2. **Interpretación semántica**: Claude Vision + GPT-4o en paralelo, para nombrar recintos, leer cuadros de superficie, contar puertas/ventanas/escaleras, y cruzar eso contra la medición geométrica independiente.
3. **Motor de reglas normativas**: cada regla (ancho mínimo de pasillo, superficie mínima, círculo de giro accesible, pendiente de rampa, etc.) está anclada a texto verificado de la OGUC/LGUC/DDU/PRC, no inventada ni parafraseada por un LLM sin ancla.
4. **Interfaz de validación gráfica del arquitecto**: el arquitecto revisa y corrige la geometría detectada antes de correr el análisis normativo completo — el sistema no se auto-engaña con datos sin validar.

El problema real que quiero resolver

El pipeline geométrico funciona razonablemente sobre UN plano de prueba, calibrado a mano durante semanas (filtros de ángulo, color, umbrales de barrido angular, tolerancias de conectividad, etc.). Cada vez que aparece un plano nuevo con una convención de dibujo distinta (otra oficina, otro software CAD, otra forma de marcar puertas/achurado), varios de esos filtros dejan de servir. Ejemplo concreto reciente: llevamos 5 rondas de diagnóstico intentando entender por qué 7 puertas reales de un plano no generan NINGÚN dato — descartamos que sean imágenes incrustadas, Form XObjects, anotaciones PDF, imágenes inline y fuentes Type3, una por una, con evidencia real en cada paso.

El desafío central, tal como lo definió el responsable del producto: que el análisis geométrico (muros, puertas, ventanas, recintos con área) sea preciso para *casi cualquier tipo de plano de arquitectura chileno/latinoamericano en PDF*, no solo el caso calibrado a mano — es decir, generalización de la extracción geométrica, no solo precisión puntual.

Lo que ya evaluamos (no lo repitas, dime qué falta)

- **Floor Plan API** (floorplanapi.com) — API REST que devuelve muros/puertas/ventanas/recintos con área desde una imagen. Quedó **bloqueada**: requiere registro propio (floorplanapi.com/register) para obtener API key, y no lo pudimos completar/obtener acceso funcional. Sigue siendo el candidato de referencia si alguien puede confirmarnos cómo conseguir acceso real, o si hay una alternativa equivalente sin esa fricción.
- **CubiCasa5K** (modelo abierto, checkpoint pre-entrenado) — probado contra nuestro ground truth: 100% precisión en ventanas pero recall bajo (29-36%), precisión muy baja en puertas (19-36%, mayoría falsos positivos correlacionados con achurado de "se construye"). Entrenado en planos residenciales finlandeses, brecha de dominio real.
- **Grounding DINO + SAM 2** (zero-shot) — 0% recall efectivo contra nuestro ground truth. Entrenado en fotografías, no reconoce símbolos de línea CAD. Confirmado con múltiples corridas, no es un problema de threshold.
- **U-Net + MLSTRUCT-FP** (Pablo Pizarro, dataset de 954 planos chilenos reales) — solo segmenta muros (no puertas/ventanas/recintos), el propio autor reporta IoU promedio 0.77 (moda 0.90) y describe su resultado como "prueba de concepto" con artefactos pendientes. Hay checkpoint pre-entrenado disponible, todavía no lo corrimos.
- **CubiCasa 2.0** (producto comercial, distinto del dataset abierto) — catalogado como "opción líder, API amigable para desarrolladores", no evaluado en profundidad todavía.
- **Togal.AI** — evaluado, descartado por precio (~US\$299/usuario/mes) y mercado objetivo (contratistas grandes de EE.UU., no estudios chilenos chicos).
- **Markovate** (soporte DWG/DXF/escaneado) y **Bild AI** (profundidad en puertas/Div. 8) — catalogados, no evaluados en profundidad.
- **Mastt** y **Kreo Caddie** — tienen patrón de chat/Q&A en lenguaje natural sobre el plano, relevante como referencia de UX, no como fuente de datos geométricos.

- MuraNet (literatura, reporta IoU 0.8 en CubiCasa5K) — sin repo público con pesos descargables, no se pudo probar.
- Faster-RCNN + ResNet — candidato de literatura, no implementado.
- Revisamos también el registro de conectores MCP de Claude buscando "floor plan/architecture/blueprint/CAD" — no existe ningún conector MCP para esto.

Lo que necesito que me respondas

1. **Herramientas o APIs concretas** (no listas genéricas de "usa OpenCV o YOLO") que extraigan geometría de planos arquitectónicos 2D (muros, puertas, ventanas, recintos con área) desde PDF o imagen rasterizada, que:
 - Tengan una vía de acceso real y verificable (API con registro que efectivamente funcione, o modelo open-source con pesos descargables) — no me interesan productos sin forma clara de probarlos.
 - Idealmente generalicen bien entre distintas convenciones de dibujo/oficinas/software CAD, no solo un dominio de entrenamiento estrecho (evita repetir el problema que ya tenemos con CubiCasa5K/DINO).
 - Si es posible, con evidencia de que funcionan razonablemente sobre planos latinoamericanos/chilenos o al menos hispanohablantes/con convenciones distintas a EE.UU./Europa — o dime explícitamente si no la hay.
2. **Papers o repos recientes (2024-2026)** de extracción/vectorización de planos con código y pesos públicos que no hayamos identificado todavía (ya conocemos MLSTRUCT-FP/Pizarro, CubiCasa5K, MuraNet, Faster-RCNN genérico — dime qué hay más allá de esto, especialmente si aborda el problema de generalización entre convenciones de dibujo, no solo precisión en un dataset fijo).
3. **Para cada herramienta que recomiendes, decime explícitamente:**
 - Cómo se accede en la práctica (API con key real, self-host, requiere GPU, etc.) y qué fricción de acceso tiene (para no repetir el bloqueo de Floor Plan API).
 - Precio o modelo de negocio si es conocido.
 - Cifras de precisión/recall/IoU solo si están publicadas con fuente verificable (paper, benchmark público, documentación técnica) — si es un claim de marketing sin evidencia, dímelo explícitamente como tal, no lo presentes con el mismo peso que un benchmark real. Ya nos pasó que un competidor (Revi) citó una fórmula normativa que resultó ser real pero mal calculada, y otro dato que resultó fabricado — priorizo evidencia verificable sobre afirmaciones de venta.
 - Si complementa o compite con lo que ya construimos (extracción vectorial determinística propia + CubiCasa5K + pendiente de correr MLSTRUCT-FP/U-Net).
4. **Una recomendación priorizada:** si tuvieras que elegir 2-3 opciones para probar primero dado que ya evaluamos las de arriba, ¿cuáles serían y por qué, específicamente pensando en el objetivo de generalización entre distintos tipos/convenciones de plano, no solo precisión en un caso?

No necesito que me expliques compliance/normativa (eso lo tenemos resuelto con nuestro propio motor de reglas) — el foco de esta consulta es exclusivamente la capa de extracción geométrica.

Sí: lo que te falta no es “otro detector genérico”, sino un **mapa de opciones realmente probables de generalizar mejor** entre convenciones de dibujo, más la distinción clara entre: extracción vectorial nativa de PDF, modelos entrenables para raster/imagen, y datasets/benchmarks nuevos que sí empujan el problema de dominio. Con lo que encontré, la mejor noticia es que hay algunas vías nuevas, pero la evidencia pública todavía es más fuerte para **muros/planos de mayor escala** que para puertas/ventanas robustas en diversidad latinoamericana. ^{[1] [2] [3] [4] [5]}

1) Herramientas / APIs concretas

Opción	Qué extrae	Acceso real	Fricción	Precio	Evidencia pública
PyMuPDF / MuPDF	Geometría vectorial PDF nativa: líneas, curvas, rectángulos, imágenes, texto	Sí, open-source	Baja; self-host	Gratis	Documentado con <code>get_drawings()</code> / <code>get_cdrawings()</code> para extraer gráficos vectoriales del PDF. ^[6]
Aspose.PDF for Python	Vector graphics del PDF, polígonos, polilíneas, paths	Sí, SDK comercial	Media; licencia y SDK	Comercial	La documentación muestra extracción de datos vectoriales y <code>GraphicsAbsorber</code> . ^{[7] [8]}
CubiCasa 2.0	Reconocimiento de planimetría desde imagen	Sí, producto comercial	Media; depende de acceso comercial	No verificado públicamente	El producto existe, pero en lo que vi no encontré benchmark técnico público suficientemente verificable para usarlo como evidencia fuerte.
DeepFloorplan / TF2DeepFloorplan	Muros, puertas/ventanas, rooms desde imagen	Sí, open-source con pesos / repos	Media; requiere setup legacy y peso preentrenado	Gratis	Repos con demo y pesos descargables; origen en papers de multi-task floor plan recognition. ^{[9] [10] [11]}
FloorPlanAnalyzer / YOLO-based repos	Detectan rooms/doors/windows	Sí, open-source	Media; calidad heterogénea	Gratis	Hay repos recientes, pero son más “engineering demos” que benchmark robusto; útil para prototipado, no como base de confianza. ^{[12] [13]}
BatchPlan	Extracción de floor plans desde IFC/BIM, no PDF 2D puro	Sí, open-source	Alta si tu entrada es solo PDF 2D	Gratis	Interesante si luego abarcas BIM/IFC, pero no compite con tu caso principal de PDF arquitectónico. ^{[14] [15]}

Qué sí recomendaría técnicamente

- **PyMuPDF** como primera capa obligatoria para PDFs vectoriales, porque ya estás haciendo lo correcto: preservar primitivos del PDF antes de rasterizar. La documentación explícitamente soporta extraer gráficos vectoriales y objetos de imagen/texto sin pasar por reconstrucción a imagen. ^[6]
- **Aspose.PDF** solo si necesitas una segunda vía comercial para comparar parsing vectorial, porque ofrece extracción de paths/polylines/polygons y GraphicsAbsorber, pero su valor es más de infraestructura que de “modelo inteligente”. ^{[7] [8]}
- **DeepFloorplan / TF2DeepFloorplan** si quieres un baseline de red multi-tarea clásica para rasterizado, con pesos y repos públicos, aunque no esperaría gran generalización sin fine-tuning fuerte a tu dominio. ^{[9] [10] [11]}

2) Papers / repos recientes útiles

Lo más relevante que encontré para tu problema de generalización es esto:

- **MSD: A Benchmark Dataset for Floor Plan Generation of Building Complexes** (ECCV 2024 / 2025 proceedings). Es importante porque explícitamente documenta el *mismatch* entre datasets actuales y “world real”, y dice que las baselines existentes no resuelven bien la complejidad de edificios/apartamentos múltiples. El repo es público y el dataset se puede descargar. ^{[4] [5]}
- **FRI-Net** (ECCV 2024). Está orientado a reconstrucción de floorplans y tiene código y pretrained models públicos. No es exactamente tu task de extracción semántica de puertas/ventanas desde PDF vectorial, pero sí es una de las piezas recientes más serias con artefactos públicos. ^[16]
- **Survey y revisiones 2025**. Hay al menos una survey seria sobre floor plan recognition que enfatiza que el área sigue teniendo problemas de generalización y razonamiento arquitectónico; no te da pesos, pero sí confirma que el “domain gap” sigue siendo el cuello de botella central. ^[17]
- **Repos recientes tipo FloorPlanAnalyzer / floor-plan-segmentation / YOLO floor-plan**. Son útiles como puntos de referencia operativos, pero no vi evidencia fuerte de benchmarks cruzados entre convenciones distintas. ^{[12] [13] [18]}
- **MLSTRUCT-FP** sigue siendo muy importante para tu caso porque es chileno y abierto; el paper y repos lo posicionan como dataset de 954 planos residenciales multi-unidad chilenos. Ya lo conocías, pero sigue siendo el único ancla local fuerte que encontré con acceso claro. ^{[2] [19] [20]}

3) Qué falta de verdad

Lo que no encontré, y esto es clave, es una **API abierta y verificable** que haga extracción geométrica robusta de planos arquitectónicos PDF 2D con buen desempeño cross-domain en Latinoamérica, tipo “sube PDF y devuelve walls/doors/windows/rooms” con acceso real, precios claros y benchmarks públicos sólidos. Floor Plan API sigue siendo la referencia conceptual, pero en la evidencia pública que pude verificar no apareció una alternativa equivalente con la misma promesa y menos fricción. ^{[5] [2] [4]}

También falta evidencia pública fuerte de que modelos open-source recientes hayan sido evaluados específicamente en **planos chilenos o hispanohablantes** con precisión/recall publicados; lo que sí aparece es que el dominio “real” multi-apartamento y los cambios de convención siguen rompiendo baselines entrenadas en datasets más estrechos. ^[4] ^[17]

4) Mi recomendación priorizada

Si yo tuviera que elegir 2-3 opciones para probar primero, iría así:

1. **MSD + FRI-Net como referencia de generalización en raster.** No porque resuelvan tu PDF vectorial directo, sino porque están entre lo más reciente y serio en datasets/modelos públicos que explícitamente atacan el gap entre datasets bonitos y planos reales complejos. Te sirven para medir cuánto de tu problema es “símbolo/convención” vs. “complejidad espacial”. ^[5] ^[16] ^[4]
2. **DeepFloorplan/TF2DeepFloorplan como baseline multi-task clásico.** Ya tiene pesos y código públicos, detecta muros/puertas/ventanas/rooms, y te da una línea base histórica para comparar. No lo veo como solución final, pero sí como un benchmark útil para ver si tu caso falla por arquitectura del modelo o por dominio. ^[10] ^[11] ^[9]
3. **Aspose.PDF o una segunda ruta de parsing vectorial como control experimental.** No para “aprender mejor”, sino para validar si el problema está en tu extracción PDF vectorial o en la interpretación posterior. Si Aspose te da caminos donde PyMuPDF no, eso te ayuda a aislar fallas de parsing sin reinventar tu motor. ^[8] ^[7]

5) Mi lectura final

Para tu objetivo, la apuesta más realista no parece ser “encontrar un modelo mágico”, sino combinar:

- **parsing vectorial nativo del PDF** muy robusto,
- **un detector raster entrenable** para símbolos ambiguos o inconsistentes,
- **y fine-tuning / adaptación al dominio chileno** con datasets locales como MLSTRUCT-FP. ^[19]
^[2] ^[6] ^[4]

Si quieres, en el siguiente paso puedo hacer una **tabla mucho más operativa**, con:

- nombre del repo/API,
- link de acceso,
- qué instalar,
- si necesita GPU,
- qué clases devuelve,
- y una recomendación de **orden de prueba** para que tu equipo lo ejecute en 1 semana sin perder tiempo.

✱✱

2. <https://repositorio.uchile.cl/bitstream/handle/2250/196842/Wall-polygon-retrieval-from-architectural-floor-plan-images-using-vectorizacion-and-Deep-Learning-methods.pdf?sequence=1&isAllowed=y>
3. <https://caspervanengelenburg.github.io/msd-eccv24-page/>
4. https://pure.tudelft.nl/ws/portalfiles/portal/230088893/978-3-031-73636-0_4.pdf
5. <https://github.com/caspervanengelenburg/msd>
6. <https://buildmedia.readthedocs.org/media/pdf/pymupdf/latest/pymupdf.pdf>
7. <https://docs.aspose.com/pdf/es/python-net/extract-vector-data-from-pdf/>
8. <https://docs.aspose.com/pdf/es/python-net/working-with-vector-graphics/>
9. <https://github.com/maikpaixao/deep-floorplan-recognition>
10. <https://github.com/zcemycl/TF2DeepFloorplan>
11. <https://deepwiki.com/zlzeng/DeepFloorplan/4.4-external-data-sources>
12. <https://github.com/mageaustralia/FloorPlanAnalyzer>
13. <https://github.com/sanatladkat/floor-plan-object-detection>
14. <https://github.com/byildiz/BatchPlan>
15. <https://repository.tudelft.nl/record/uuid:2899ea54-d769-4154-bb04-3c95b018a194>
16. <https://github.com/Daisy-1227/FRI-Net>
17. <https://dl.acm.org/doi/full/10.1145/3747227.3747250>
18. <https://github.com/ozturkoktay/floor-plan-room-segmentation>
19. <https://github.com/MLSTRUCT/MLStructFP>
20. <https://github.com/MLSTRUCT/MLSTRUCT-FP>
21. <https://stackoverflow.com/questions/72848330/how-to-read-pdf-images-as-opencv-images-using-pymupdf>
22. <https://pymupdf.readthedocs.io/en/latest/tutorial.html>
23. <https://www.educative.io/courses/pdf-management-python/pinpointing-the-differences-between-two-pdf-files>
24. <https://pymupdf.io/>
25. <https://github.com/pymupdf/PyMuPDF>
26. <https://revistainvestigato.com/EditorInvestigo/index.php/hm/article/download/433/A20P/1411>
27. <https://www.statical.eu/es/documentacion-de-reparacion-de-muros/>
28. <https://ithy.com/article/pdf-iges-geometry-extraction-05y3roe0>
29. <https://medium.com/@bocharova.maiia/scanned-pdfs-quality-enhancement-using-pymupdf-opencv-d2a291a0d822>
30. https://repositoriosdigitales.sicyt.gob.ar/vufind/Record/RDUUNC_39ed61b7910cbe91f4d61ca823267ce4
31. https://arquitecturaacademica.files.wordpress.com/2020/12/volumen_6_tomo_v_muros.pdf
32. <https://arxiv.org/pdf/2607.18997.pdf>
33. <https://github.com/topics/floorplan?l=python&o=asc&s=updated>
34. <https://github.com/RasterScan/Floor-Plan-Recognition>
35. <https://github.com/nickorzha/architecture-floorplan-automatic>
36. <https://github.com/whchien/deep-floor-plan-recognition>
37. <https://huggingface.co/rikhoffbauer2/floorplan-parser>

38. <https://www.mdpi.com/2673-2688/6/4/67>
39. <https://archshaper.com/architecture-line-weight-guide>
40. <https://deepwiki.com/whchien/deep-floor-plan-recognition/5-ml-model>
41. <https://gist.github.com/shubhamwagh/3c7db4fd1e2f87f9139343d523343dfb>
42. <https://universe.roboflow.com/university-y9nbi/floor-plans-500>
43. <https://pure.tudelft.nl/ws/portalfiles/portal/246888401/mostafavi-et-al-2024-floor-plan-generation-the-interplay-among-data-machine-and-designer.pdf>
44. <https://github.com/Beemeral/ROBIN-PlanFloorsDataset>
45. <https://github.com/MLSTRUCT>
46. <https://pdfs.semanticscholar.org/558c/fec5a854a8c0e775c25bab2b8589097ce170.pdf>
47. <https://github.com/zlzeng/DeepFloorplan/issues>
48. <https://gesstalt.github.io/research/floorplan.html>
49. <https://github.com/Dzhuhnuhmeidzhai/FloorPlanResearcher>
50. https://github.com/nestauk/asf_floorplan_interpreter
51. <https://github.com/singhmanmeetsingh/CUBICASA>
52. <https://github.com/patnicolas/floorplan>
53. <https://github.com/zlzeng/DeepBuildingPlanes>
54. <https://carbonimage.github.io/news/batch-plan>
55. <http://arxiv.org/pdf/1710.01802.pdf>
56. <https://www.preprints.org/manuscript/202501.1553/v1>
57. <https://www.sciencedirect.com/science/article/pii/S0926580525004182>
58. <https://www.diva-portal.org/smash/get/diva2:1660783/FULLTEXT01.pdf>
59. <https://github.com/spatialxia/Tell2Floorplan-dataset>
60. <https://apify.com/ntriquipro/blueprint-intelligence>
61. <http://refbase.cvc.uab.es/files/HSL2013.pdf>
62. <https://universe.roboflow.com/floor-plan-recognition-research/floor-plan-wnhb5>